

Relatório de Implementação

Este documento conta **como** o projeto foi construído: as escolhas que fizemos, como cada parte funciona por dentro, e como usar.

1) O que a Lang é

A **Lang** é uma linguagem imperativa simples, com tipos estáticos e foco didático. Ela tem:

- **Primitivos:** Int, Float, Bool, Char.
 - **Compostos:** vetores `T[ ]` e produto `A * B`.
 - **Expressões:** aritméticas, relacionais e booleanas com curto-circuito.
 - **Comandos:** atribuição, `if/then[/else]`, `while`, `iterate(...)` `{ ... }`, `print`, `read`, blocos `{ ... }`.
 - **Literais:** inteiros, floats, chars, booleanos, null, vetores `[1, 2, 3]` e strings (úteis para depuração).
-

2) Visão geral do pipeline

A implementação segue o “fluxo clássico” de compiladores, com separação clara de responsabilidades:

Léxico e Sintaxe (ANTLR)

As gramáticas definem os tokens e as regras de parsing. Optamos por uma gramática **limpa e próxima do enunciado**, com uma regra explícita para `iterate`:

```
iterateCmd : ITERATE '(' (lvalue ':' expr | expr) ')' cmd ;
```

1. Com isso, suportamos os dois modos:

- `iterate(i:N) { ... }` para contagem

- `iterate(x:arr) { ... }` para varrer coleções

2. **AST e construção de nós (Builder)**

A árvore sintática abstrata (AST) mora em `lang.ast`. O `AstBuilder` transforma a parse tree do ANTLR em nós de AST **enxutos**, por exemplo:

- Expressões: `IntLit`, `FloatLit`, `Var`, `BinOp`, `UnOp`, `Index`, `ArrayLit`
- Comandos: `Assign`, `If`, `While`, `Iterate`, `Print`, `Read`, `Block`
- Tipos: `Int`, `Float`, `Bool`, `Char`, `Array(T)`, `Product(A, B)`, `Named(TYID)`

3. A decisão aqui foi **não complicar a AST**: cada nó guarda o essencial (filhos, operador e posição). Isso simplifica todos os visitantes.

4. **Verificação de tipos (Type Checker)**

O `TypeChecker` percorre a AST, resolve símbolos e checa compatibilidades:

- Guardas de `if/while` devem ser `Bool`
- `iterate(i:N)` exige `N: Int`
- Indexação `a[i]` exige `a: T[]` e `i: Int`
- Atribuição verifica tipo do RHS contra o LHS
- `print` e `read` tratadas como I/O tipado

5. **Interpretador (execução direta)**

O `Interpreter` avalia a AST usando um modelo de valores (`Int`, `Float`, `Bool`, `Char`, `String`, `Array`, `Null`) e ambientes de variáveis.

Aqui lidamos com **erros de runtime** de forma clara:

- Divisão ou módulo por zero
- Índice fora de faixa
- Guarda não booleana em `if/while`
- Ausência de `main`

6. O interpretador é a **fonte da verdade semântica** do projeto. Tudo que geramos deve reproduzir o mesmo comportamento.

7. **Geração *source-to-source***

Existem dois:

- **PrettyPrinter**: reimprime Lang de forma padronizada. É ótimo para depurar e checar a AST.
- **JavaGen**: gera **Java** equivalente. Com ele, atendemos a ideia de “código de alto nível alvo”.

8. **Geração de baixo nível (Jasmin/JVM)**

O JasminGen produz assembly Jasmin. Cuidamos para que a semântica seja a mesma do interpretador.

Para garantir isso, usamos **testes diferenciais**: rodamos o interpretador, geramos Jasmin, montamos e executamos, e comparamos as saídas. Se divergir, há bug no codegen.

CLI simples e direto

A interface de linha de comando foi pensada para ser simples:

```
java -jar lang.jar <flag> arquivo.lang
```

9. Flags planejadas:

- `-syn` valida sintaxe
- `-t` roda o type checker
- `-i` interpreta
- `-src` gera alto nível (PrettyPrinter ou Java)
- `-gen` gera Jasmin

10. Se na sua cópia apenas `-i` estiver ligado, é só **plugar os outros cases** no `switch(flag)` do CLI. Os componentes por trás já existem.
-

3) Como pensamos as estruturas

- **AST minimalista:** nós claros, sem metadados supérfluos. Resultado: visitantes simples, menos risco de inconsistências.
 - **Símbolos e escopos:** uma tabela por escopo, com resolução direta de identificadores. Evitamos “mágica” e priorizamos legibilidade.
 - **Tipagem pragmática:** regras diretas e previsíveis. Exemplo:
 - `Int + Int → Int`
 - `Float + Int → Float`
 - `a[i]` devolve o tipo do elemento de `a`
 - `if` e `while` exigem `Bool`
 - **Interpretação como verdade:** tudo é validado contra o que o interpretador faz. Isso reduz o custo de manter dois mundos diferentes.
 - **Dois backends de alto nível:** `PrettyPrinter` ajuda a entender a AST, Java atende o pedido de “código alvo”.
 - **Jasmin com testes diferenciais:** não basta “gerar”. É preciso ter o **mesmo comportamento** que o interpretador. Essa verificação fecha o ciclo.
-

4) Exemplos práticos da linguagem

4.1 Condicional simples

```
main() {  
  x = 10; y = 5;  
  if x > y then print x + y else print x - y  
}
```

- `-syn: accept`

- -t: accept
- -i: imprime 15
- -src: gera código equivalente (PrettyPrinter ou Java)
- -gen: emite Jasmin; executando, também imprime 15

4.2 iterate por contagem

```
main() {
  iterate(i:3) {
    print i    {- 0, 1, 2 -}
  }
}
```

4.3 Vetores e indexação

```
main() {
  v = [1,2,3];
  v[1] = 99;
  print v[1]  {- 99 -}
}
```

4.4 Curto-circuito lógico

```
main() {
  a = 0;
  if (false && (a = 1) == 1) then print 1 else print a
  {- imprime 0; o lado direito não é avaliado -}
}
```

5) Como usar no terminal

Build do fat JAR
mvn clean package

Sintaxe
java -jar target/lang-*-jar-with-dependencies.jar -syn prog.lang

Tipos

```
java -jar target/lang-*-jar-with-dependencies.jar -t prog.lang
```

Interpretar

```
java -jar target/lang-*-jar-with-dependencies.jar -i prog.lang
```

Source-to-source (escolha PrettyPrinter ou Java no CLI)

```
java -jar target/lang-*-jar-with-dependencies.jar -src prog.lang > out.java
```

Jasmin (baixo nível)

```
java -jar target/lang-*-jar-with-dependencies.jar -gen prog.lang > out.j
```

Para testar geração e execução via Jasmin, é necessário ter o JASMIN_JAR configurado no ambiente.

6) Conclusão

O projeto entrega um pipeline completo, com gramática, AST, tipos, interpretação e dois níveis de geração de código. As decisões priorizaram **clareza**, **modularidade** e **equivalência de comportamento** entre intérprete e código gerado. Com o CLI totalmente ligado e o ajuste fino opcional de I/O, a entrega fica redonda do ponto de vista técnico e didático.

Fontes bibliográficas e materiais de apoio

- **ANTLR (site oficial)** — visão geral, downloads e materiais. (antlr.org)
- **ANTLR v4 (documentação no GitHub)** — referências de gramática, exemplos e guias. ([GitHub](https://github.com/antlr/antlr4))
- **JVM Specification (SE 22, índice)** — estrutura da JVM, class file, verificação, instruções. ([Oracle Docs](https://docs.oracle.com/javase/22/index.html))
- **JVM Specification — Capítulo 6 (Instruções da JVM)** — referência das instruções bytecode. ([Oracle Docs](https://docs.oracle.com/javase/22/notes/index.html))
- **Jasmin (site oficial)** — introdução e downloads do montador para JVM. ([Jasmin](https://jasmin.sourceforge.net/))
- **Jasmin User Guide** — guia de uso e sintaxe do assembler. ([Jasmin](https://jasmin.sourceforge.net/))